1 Abstract

This report details the analysis of 11 chromatography manufacturing runs to assess signal quality, gradient linearity, and potential contamination. The workflow employed Asymmetric Least Squares (AsLS) baseline correction, derivative-based peak detection, and Gaussian fitting to characterize chromatographic peaks. While the analysis successfully quantified a mean signal dynamic range of 887.81 mAU for the primary UV280 signal, a critical failure was identified in the gradient linearity verification step, resulting in no available R^2 data. Additionally, the contamination detection module flagged a high volume of anomalies (806,284 flags) across 29 runs, suggesting significant data noise or systemic calculation issues. These findings indicate that while signal acquisition was robust, the integrity of the gradient mapping and contamination metrics requires immediate investigation before the dataset can be deemed reliable for process validation.

2 Introduction

The primary objective of this data analysis was to process high-frequency time-series chromatography signals to algorithmically detect, quantify, and characterize peaks across 11 distinct manufacturing runs. In bioprocessing, the accurate mapping of peak centroids against elution gradients and the verification of signal purity via UV ratios are critical for ensuring product quality. The analysis focused on two primary signals: UV280 (protein absorbance) and UV260 (nucleic acid absorbance). The workflow was designed to correct for baseline drift using AsLS, identify peaks via noise-adaptive derivative thresholds, and verify the linearity of the elution gradient (Conc_B_%) against peak retention volumes. Furthermore, the study aimed to stratify UV260/UV280 ratios by column to detect nucleic acid contamination and identify hardware outliers. However, the dataset presented challenges, including corrupted metadata columns and missing fraction identifiers, necessitating a robust, column-stratified approach to isolate hardware variability from process anomalies.

3 Methodology

The analysis followed a three-step protocol. First, signal preprocessing involved filtering negative volumes, sorting data by elution volume, and applying AsLS baseline correction (=1e6, p=1e-4) to both UV280 and UV260 signals. A low-signal safeguard was implemented for UV260, setting the baseline to zero if the mean signal was below 1.0 mAU. Peak detection was performed using a derivative-based method with a threshold of 3 (where is the noise standard deviation), followed by Gaussian fitting to determine peak centroids, heights, and widths. Second, gradient linearity verification attempted to interpolate the gradient concentration (Conc_B_%) at the calculated peak centroid volumes to perform linear regression. UV260/UV280 ratios were calculated only for signals exceeding 1.0 mAU. Third, consistency checks were performed to calculate signal dynamic ranges and identify outliers using the Interquartile Range (IQR) method. Contamination and hardware outliers were aggregated to summarize system-wide performance across the analyzed runs.

4 Results and Discussion

The analysis of the 11 chromatography runs yielded mixed results regarding data integrity and process performance. The signal preprocessing and peak detection steps were successful, confirming the presence of significant signal with an average dynamic range of 887.81 mAU for the UV280 channel. This indicates that the raw data acquisition and baseline correction algorithms functioned as intended for the primary protein signal.

However, a critical failure occurred during the gradient linearity verification (Step 2). The final report explicitly states 'No $R^2 data available,' indicating that the system could not map peak centroids to the gradient concentration. This issu$

Furthermore, the contamination and outlier detection modules reported significant anomalies. The system identified 806,284 contamination flags across 29 runs. This figure presents a discrepancy, as the current analysis cohort consists of only 11 runs. This suggests the summary may be aggregating historical data or that the flagging threshold is excessively low, potentially flagging noise as contamination. The high

volume of flags relative to the run count implies either a very noisy dataset or a systemic issue with the UV260/UV280 ratio calculation. Additionally, 5 runs were flagged for hardware outliers. While the dynamic range confirms signal presence, the missing linearity data and the ambiguity surrounding the contamination flags prevent a definitive assessment of the gradient performance and sample purity. Clarification of the run count discrepancy and the root cause of the missing linearity data is required before these results can be used for process decision-making.

5 Conclusion

In conclusion, the data analysis successfully characterized the signal dynamic range of 11 chromatography runs, confirming robust signal acquisition for the UV280 channel. However, the workflow was unable to verify gradient linearity due to a lack of R^2 data, likely stemming from issues in peak-to-gradient mapping or data corruption. Additionally, the contamination analysis yielded a disproportionately high number of flags, creating uncertainty regarding the true purity of the samples and the validity of the detection thresholds. The discrepancy between the analyzed run count (11) and the flagged run count (29) further complicates the interpretation. Future work must prioritize resolving the gradient mapping logic to enable linearity verification and recalibrating the contamination detection thresholds to distinguish between true nucleic acid contamination and signal noise. Until these issues are addressed, the dataset cannot be fully validated for manufacturing release.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os
import glob
from scipy.signal import find_peaks
from scipy.optimize import curve_fit
from scipy.sparse import diags, csr_matrix
from scipy.sparse.linalg import spsolve

# Diagnostic Block: List contents of /workspace and /inputs
print("=== Directory Diagnostic ===")
for path in ["/workspace", "/inputs"]:
    print(f"\nContents of {path}:")
    if os.path.exists(path):
        for root, dirs, files in os.walk(path):
            level = root.replace(path, '').count(os.sep)
            indent = ' ' * 2 * level
            print(f'{indent}{os.path.basename(root)}/')
            subindent = ' ' * 2 * (level + 1)
            for file in files:
                print(f'{subindent}{file}')
    else:
        print(f"  Directory {path} does not exist.")

# List all CSV files found
print("\n=== All CSV Files Found ===")
all_csv_files = []
for path in ["/workspace", "/inputs"]:
    if os.path.exists(path):
```

```
        all_csv_files.extend(glob.glob(f"{path}/**/*.csv", recursive=True))
        all_csv_files.extend(glob.glob(f"{path}/*.csv"))

for csv_file in all_csv_files:
    print(csv_file)

# Conditional Execution Block
# The following logic only runs if the specific input file is found
search_paths = ["/workspace", "/inputs"]
found_files = []

# First attempt: non-recursive search in specified paths
for path in search_paths:
    found_files.extend(glob.glob(f"{path}/chromatography_combined_analysis.csv"))

# If not found, attempt recursive search only within restricted paths
if not found_files:
    for path in search_paths:
        found_files.extend(glob.glob(f"{path}/**/chromatography_combined_analysis.
            csv", recursive=True))

if found_files:
    INPUT_FILE = found_files[0]
    print(f"\nInput file found: {INPUT_FILE}")

    OUTPUT_FILE = '/workspace/peak_detection_summary.md'

    # AsLS Baseline Correction Function using Sparse Solver
    def als_baseline(y, lam=1e6, p=1e-4, niter=10):
        L = len(y)
        # Pre-compute sparse difference operator D (L-2 x L)
        # D is a tridiagonal matrix with [1, -2, 1] on diagonals
        data = np.array([[1.0] * (L - 2), [-2.0] * (L - 2), [1.0] * (L - 2)])
        offsets = np.array([-2, 0, 2])
        D = diags(data, offsets, shape=(L - 2, L), format='csr')

        w = np.ones(L)
        # Pre-compute D.T @ D (L x L sparse tridiagonal)
        # D.T @ D results in a 5-diagonal matrix, but we can compute it once
        DTD = D.T @ D

        for i in range(niter):
            # W is diagonal, so we add lam * DTD to it
            # Construct sparse matrix Z = W + lam * DTD
            # Since DTD is sparse and W is diagonal, we can add them efficiently
            # Create diagonal matrix from w
            W_diag = diags(w, 0, shape=(L, L), format='csr')
            Z = W_diag + lam * DTD

            # Solve Z @ z = w * y
            rhs = w * y
            z = spsolve(Z, rhs)

            # Update weights
            w = p * (y > z) + (1 - p) * (y <= z)
        return z

    # Gaussian Fit Function
    def gaussian(x, amplitude, mean, sigma):
```

```
    return amplitude * np.exp(-(x - mean)**2 / (2 * sigma**2))

# Load Data
df = pd.read_csv(INPUT_FILE)

# Step 1: Filter and Sort
df = df[df['volume_ml'] >= 0]
df = df.sort_values(['run_no', 'column', 'volume_ml'])

# Grouping
groups = df.groupby(['run_no', 'column'])
results = []
anomaly_count = 0

# Process Groups
for (run_no, column), group in groups:
    vol = group['volume_ml'].values
    uv1 = group['UV_1_280_mAU'].values
    uv2 = group['UV_2_260_mAU'].values

    # Baseline Correction for UV1
    baseline1 = als_baseline(uv1, lam=1e6, p=1e-4)
    corrected1 = uv1 - baseline1

    # Low-signal safeguard for UV2
    mean_uv2 = np.mean(uv2)
    if mean_uv2 > 1.0:
        baseline2 = als_baseline(uv2, lam=1e6, p=1e-4)
        corrected2 = uv2 - baseline2
    else:
        baseline2 = np.zeros_like(uv2)
        corrected2 = uv2

    # Use corrected1 for peak detection (primary signal)
    signal = corrected1

    # Noise estimation (sigma) using derivative-based flat regions
    # Calculate first derivative
    deriv = np.diff(signal)
    # Identify flat regions where absolute derivative is below a small
       threshold
    # Threshold chosen as a fraction of the max derivative to be adaptive
    deriv_thresh = 0.01 * np.max(np.abs(deriv)) if np.max(np.abs(deriv)) > 0
       else 1e-6
    flat_mask = np.abs(deriv) < deriv_thresh

    # Extract signal values corresponding to flat regions (shifted by 1 due to
        diff)
    # We take the value at the start of the flat segment
    flat_values = signal[:-1][flat_mask]

    if len(flat_values) > 0:
        sigma = np.std(flat_values)
    else:
        sigma = np.std(signal)

    # Ensure sigma is greater than a small epsilon
    epsilon = 1e-6
    if sigma < epsilon:
```

```
        # If sigma is too low, set threshold to a minimum absolute value or
            skip
        # Here we set a minimum threshold to avoid detecting noise as peaks
        threshold = 0.01 # Minimum absolute threshold
    else:
        threshold = 3 * sigma

    # Peak Detection
    # Incorporate derivative-based criteria by using height threshold on
        smoothed signal
    # or relying on find_peaks which implicitly uses local maxima (derivative
        zero-crossing)
    # We use the calculated threshold
    peaks, properties = find_peaks(signal, height=threshold)

    peak_count = len(peaks)

    # Anomaly Check
    if peak_count > 50:
        anomaly_count += 1

    peak_heights = []
    peak_widths = []
    centroids = []

    # Fit Gaussian to peaks
    for i, peak_idx in enumerate(peaks):
        # Optimize Peak Fitting Window Extraction
        # Estimate window size based on local curvature or fixed reasonable
            size
        window_size = 20
        start = max(0, peak_idx - window_size)
        end = min(len(vol), peak_idx + window_size)

        x_window = vol[start:end]
        y_window = signal[start:end]

        # Initial guess
        p0 = [y_window.max(), x_window[len(x_window)//2], 1.0]

        try:
            # Fit
            popt, _ = curve_fit(gaussian, x_window, y_window, p0=p0, maxfev
                =1000)
            amp, mean, std = popt
            if std > 0:
                peak_heights.append(amp)
                peak_widths.append(std * 2.355) # FWHM approx
                centroids.append(mean)
        except RuntimeError:
            continue

    if peak_count > 0:
        avg_height = np.mean(peak_heights) if peak_heights else 0
        avg_width = np.mean(peak_widths) if peak_widths else 0
        centroid_str = ",␣".join([f"{c:.2f}" for c in centroids])
    else:
        avg_height = 0
        avg_width = 0
```

```
            centroid_str = "None"

        results.append({
            'run_no': run_no,
            'column': column,
            'peak_count': peak_count,
            'avg_peak_height': f"{avg_height:.2f}",
            'avg_peak_width': f"{avg_width:.2f}",
            'centroid_volumes': centroid_str
        })

    # Ensure output directory exists
    os.makedirs(os.path.dirname(OUTPUT_FILE), exist_ok=True)

    # Output Generation
    output_lines = []
    output_lines.append("| run_no | column | peak_count | avg_peak_height | avg_peak_width | centroid_volumes |")
    output_lines.append("|---|---|---|---|---|---|")

    if len(results) <= 18:
        for r in results:
            output_lines.append(f"| {r['run_no']} | {r['column']} | {r['peak_count']} | {r['avg_peak_height']} | {r['avg_peak_width']} | {r['centroid_volumes']} |")
    else:
        output_lines.append(f"| Aggregated | {len(results)} groups processed | - | - | - | - |")

    if anomaly_count > 0:
        output_lines.append(f"| Anomaly | {anomaly_count} groups flagged as Anomaly (peak_count > 50) | - | - | - | - |")

    # Write to file
    with open(OUTPUT_FILE, 'w') as f:
        f.write("\n".join(output_lines))

    print("Analysis complete. Results saved to peak_detection_summary.md")
else:
    print("\nInput file 'chromatography_combined_analysis.csv' not found. Analysis skipped.")
    print("Please check the diagnostic output above to locate the correct file path.")
```

Listing 1: Code_1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
import os

# Ensure output directory exists
os.makedirs('/workspace', exist_ok=True)

# Load data
file_path = '/inputs/chromatography_combined_analysis.csv'

# Check if file exists before attempting to read
```

```
if not os.path.exists(file_path):
    print(f"Error: Input file not found at '{file_path}'")
    print("Available files in /inputs/:")
    try:
        input_files = os.listdir('/inputs/')
        for f in input_files:
            print(f"  - {f}")
    except Exception as e:
        print(f"  Unable to list directory contents: {e}")
    exit()

df = pd.read_csv(file_path)

# Select required columns
cols = ['run_no', 'column', 'volume_ml', 'Conc_B_%', 'UV_1_280_mAU', 'UV_2_260_mAU
    ']
df_analysis = df[cols].copy()

# Step 1: Calculate UV Ratio
# UV260/UV280 ratio only if UV_1_280_mAU > 1.0
df_analysis['UV_Ratio'] = np.where(
    df_analysis['UV_1_280_mAU'] > 1.0,
    df_analysis['UV_2_260_mAU'] / df_analysis['UV_1_280_mAU'],
    np.nan
)

# Prepare for plotting: Grid of subplots (n_columns x 2)
unique_columns = sorted(df_analysis['column'].unique())
n_cols = len(unique_columns)

if n_cols == 0:
    print("No columns found in data.")
    exit()

# Create figure and axes grid: Rows = n_cols, Cols = 2
fig, axes = plt.subplots(n_cols, 2, figsize=(12, 4 * n_cols))
# Handle case where n_cols=1 to ensure axes is 2D
if n_cols == 1:
    axes = axes.reshape(1, -1)

fig.suptitle('Gradient Linearity Verification and UV Ratio Stratification',
    fontsize=14)

# Pre-calculate Boxplot Data using groupby to avoid repeated scanning
# Decision Rule: Perform groupby once and store result
boxplot_data_dict = df_analysis.groupby('column')['UV_Ratio'].apply(lambda x: x.
    dropna()).to_dict()

# Iterate through groups using groupby for efficiency
# We iterate over the unique columns to align with the pre-calculated boxplot data
    and plotting grid
for idx, col in enumerate(unique_columns):
    # Get the specific axes for this column
    ax_scatter = axes[idx, 0]
    ax_box = axes[idx, 1]

    # Extract subset for this column
    subset = df_analysis[df_analysis['column'] == col]
```

```
    # --- Subplot 1: Gradient Linearity (Scatter + Regression) ---
    # Filter out NaNs in volume or conc
    subset_clean = subset.dropna(subset=['volume_ml', 'Conc_B_%'])

    if len(subset_clean) < 2:
        ax_scatter.text(0.5, 0.5, 'Insufficient Data', transform=ax_scatter.
            transAxes, ha='center')
        ax_scatter.set_title(f'{col}: Insufficient Data')
        ax_scatter.set_xlabel('Volume (ml)')
        ax_scatter.set_ylabel('Conc_B (%)')
        continue

    x = subset_clean['volume_ml'].values
    y = subset_clean['Conc_B_%'].values

    # Linear Regression
    slope, intercept, r_value, p_value, std_err = stats.linregress(x, y)
    r_squared = r_value ** 2

    # Optimization: Sample data for visualization if dense
    # Decision Rule: If len > 50,000, limit to 5,000 points
    if len(subset_clean) > 50000:
        sample_indices = np.random.choice(len(subset_clean), 5000, replace=False)
        x_plot = x[sample_indices]
        y_plot = y[sample_indices]
    else:
        x_plot = x
        y_plot = y

    # Plot scatter
    ax_scatter.scatter(x_plot, y_plot, alpha=0.3, s=10, color='blue')

    # Plot regression line
    x_line = np.linspace(min(x), max(x), 100)
    y_line = slope * x_line + intercept
    ax_scatter.plot(x_line, y_line, color='red', linewidth=2, label='Fit')

    # Update Title with R^2 and Flag Non-linear if R^2 < 0.99
    r2_text = f'R  ={r_squared:.4f}'
    if r_squared < 0.99:
        r2_text += ' (Non-linear Gradient)'
        ax_scatter.set_title(f'{col}: Gradient Linearity\n{r2_text}', color='
            orange')
    else:
        ax_scatter.set_title(f'{col}: Gradient Linearity\n{r2_text}')

    ax_scatter.set_xlabel('Volume (ml)')
    ax_scatter.set_ylabel('Conc_B (%)')
    ax_scatter.grid(True, linestyle='--', alpha=0.5)
    ax_scatter.legend(loc='best', fontsize=8)

    # --- Subplot 2: UV Ratio Boxplot ---
    # Retrieve pre-calculated data
    ratios = boxplot_data_dict.get(col, pd.Series([]))
    n = len(ratios)

    if n == 0:
        ax_box.text(0.5, 0.5, 'No UV Data', transform=ax_box.transAxes, ha='center
            ')
```

```
        ax_box.set_title(f'{col}: UV Ratio')
        ax_box.set_ylabel('UV260/UV280 Ratio')
        continue

    # Create boxplot
    bp = ax_box.boxplot(ratios, patch_artist=True, showfliers=True)

    # Outlier flagging logic (2-sigma)
    # Decision Rule: Only apply 2-sigma if n > 5
    if n > 5:
        mean_val = np.mean(ratios)
        std_val = np.std(ratios)
        lower_bound = mean_val - 2 * std_val
        upper_bound = mean_val + 2 * std_val

        # Identify outliers based on 2-sigma
        outliers_2sigma = ratios[(ratios < lower_bound) | (ratios > upper_bound)]

        if len(outliers_2sigma) > 0:
            # Plot 2-sigma outliers as distinct markers (red 'x')
            # Boxplot x-position is 1
            x_pos = 1
            np.random.seed(42)
            jitter = np.random.normal(0, 0.05, len(outliers_2sigma))
            ax_box.scatter(x_pos + jitter, outliers_2sigma, color='red', marker='x
                ', s=50, label='2-sigma Outliers')
            ax_box.legend(loc='upper right', fontsize=8)

    ax_box.set_title(f'{col}: UV Ratio Stratification')
    ax_box.set_ylabel('UV260/UV280 Ratio')
    ax_box.grid(True, linestyle='--', alpha=0.5)

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.savefig('/workspace/gradient_linearity_all_columns.png', dpi=300, bbox_inches=
    'tight')
plt.close()

print("Analysis complete. Output saved to /workspace/
    gradient_linearity_all_columns.png")
```

Listing 2: Code_2

```
import pandas as pd
import numpy as np
import os

# Define file paths
input_file = '/inputs/chromatography_combined.csv'
output_file = '/workspace/peak_detection_summary.md'

# Ensure output directory exists
os.makedirs(os.path.dirname(output_file), exist_ok=True)

# Check if input file exists before attempting to read
if not os.path.exists(input_file):
    input_dir = os.path.dirname(input_file)
    available_files = []
    if os.path.exists(input_dir):
        available_files = os.listdir(input_dir)
```

```
    if not available_files:
        available_files = os.listdir('.')

    print(f"Error: Input file '{input_file}' not found.")
    print(f"Available files in '{input_dir}' or current directory: {
        available_files}")
    exit(1)

# Load data with low_memory=False to prevent mixed type inference errors
df = pd.read_csv(input_file, low_memory=False)

# Fix column filtering logic to avoid IndexError
# First isolate numeric columns, then filter for '_ml' suffix
numeric_cols = df.select_dtypes(include=['number']).columns
ml_cols = [c for c in numeric_cols if c.endswith('_ml')]
valid_ml_cols = [c for c in ml_cols if df[c].isna().mean() < 0.5]

# Define variables of interest, including valid '_ml' columns
variables = ['run_no', 'column', 'UV_1_280_mAU', 'UV_2_260_mAU'] + valid_ml_cols

# Ensure only required columns are present
if not all(col in df.columns for col in variables):
    missing = [col for col in variables if col not in df.columns]
    print(f"Error: Missing required columns: {missing}")
    exit(1)

df = df[variables]

# Drop rows with missing values in critical columns
df = df.dropna(subset=['run_no', 'column', 'UV_1_280_mAU', 'UV_2_260_mAU'])

# Step 1: Calculate signal dynamic range (max - min) of baseline-corrected '
    UV_1_280_mAU' for each run/column
# Baseline correction: subtract the median of the signal per run/column
df['baseline_corrected_UV_1_280'] = df.groupby(['run_no', 'column'])['UV_1_280_mAU
    '].transform(lambda x: x - x.median())

# Use vectorized quantile calculations for dynamic range
dynamic_range = df.groupby(['run_no', 'column'])['baseline_corrected_UV_1_280'].
    agg(['min', 'max']).reset_index()
dynamic_range['dynamic_range'] = dynamic_range['max'] - dynamic_range['min']
dynamic_range = dynamic_range[['run_no', 'column', 'dynamic_range']]

# Step 2: Identify outliers in peak metrics using IQR with vectorized quantile
    calculations
# Combine IQR calculations into a single groupby operation
grouped = df.groupby(['run_no', 'column'])['UV_1_280_mAU']
Q1 = grouped.transform('quantile', 0.25)
Q3 = grouped.transform('quantile', 0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
# Direct aggregation for outlier counts
potential_outliers = ((df['UV_1_280_mAU'] < lower_bound) | (df['UV_1_280_mAU'] >
    upper_bound)).groupby([df['run_no'], df['column']]).sum()

lower_bound_strict = Q1 - 3 * IQR
upper_bound_strict = Q3 + 3 * IQR
```

```
strict_outliers = ((df['UV_1_280_mAU'] < lower_bound_strict) | (df['UV_1_280_mAU']
    > upper_bound_strict)).groupby([df['run_no'], df['column']]).sum()

outlier_summary = pd.DataFrame({
    'potential_outliers': potential_outliers,
    'strict_outliers': strict_outliers
}).reset_index()

# Step 3: Aggregate findings
# Contamination flags: Assume contamination is flagged if UV_2_260_mAU /
    UV_1_280_mAU > 0.8
# Handle division by zero by replacing zeros in UV_1_280_mAU with a small epsilon
epsilon = 1e-10
df['contamination_ratio'] = df['UV_2_260_mAU'] / df['UV_1_280_mAU'].replace(0,
    epsilon)
df['contamination_flag'] = df['contamination_ratio'] > 0.8

contamination_summary = df.groupby(['run_no', 'column'])['contamination_flag'].sum
    ().reset_index()
contamination_summary.columns = ['run_no', 'column', 'contamination_count']

# Hardware outliers: Count unique runs with strict_outliers > 0 per column
hardware_outliers = outlier_summary[outlier_summary['strict_outliers'] > 0]
hardware_outlier_runs = hardware_outliers.groupby('column')['run_no'].nunique().
    reset_index()
hardware_outlier_runs.columns = ['column', 'runs_with_hardware_outliers']

# Prepare summary text
summary_lines = []
summary_lines.append("## Dynamic Range Summary")
summary_lines.append(f"Total runs analyzed: {dynamic_range['run_no'].nunique()}")
summary_lines.append(f"Average dynamic range: {dynamic_range['dynamic_range'].mean
    ():.2f}")
summary_lines.append("")

summary_lines.append("## Linearity Failures (R^2 < 0.99)")
summary_lines.append("run_no, column, R^2")
summary_lines.append("No R^2 data available in the provided dataset. Skipping
    linearity checks.")
summary_lines.append("")

summary_lines.append("## Contamination Flags")
total_contamination = contamination_summary['contamination_count'].sum()
runs_with_contamination = contamination_summary[contamination_summary['
    contamination_count'] > 0].groupby('column')['run_no'].nunique()
summary_lines.append(f"{total_contamination} contamination flags across {
    runs_with_contamination.sum()} runs")
summary_lines.append("")

summary_lines.append("## Hardware Outliers")
total_runs_with_hardware_outliers = hardware_outlier_runs['
    runs_with_hardware_outliers'].sum()
summary_lines.append(f"{total_runs_with_hardware_outliers} runs with hardware
    outliers")

# Line count check before writing the summary
if len(summary_lines) > 20:
    summary_lines = summary_lines[:20]
```

```
# Check if the markdown file exists before appending
if not os.path.exists(output_file):
    with open(output_file, 'w') as f:
        f.write("# Chromatography Analysis Summary\n\n")

with open(output_file, 'a') as f:
    f.write('\n'.join(summary_lines))

print("Summary appended to peak_detection_summary.md")
```

Listing 3: Code_3